

Towards High-Goodput LLM Serving with Prefill-decode Multiplexing

Yukang Chen*

chenyukang@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Weihao Cui*

weihao@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China
National University of Singapore
Singapore

Han Zhao*

zhao-han@cs.sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Ziyi Xu

xzy2022@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Xiaozhe Fan

jasonfxz@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Xusheng Chen

michael.xschen@gmail.com
Researcher
Shanghai, China

Yangjie Zhou

yj_zhou@nus.edu.sg
National University of Singapore
Singapore

Shixuan Sun

sunshixuan@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Bingsheng He

dcsheb@nus.edu.sg
National University of Singapore
Singapore

Quan Chen[†]

chen-quan@cs.sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Abstract

Large Language Model (LLM) serving must meet stringent Service Level Objectives (SLOs) for both the prefill and decode phases. Some existing solutions disaggregate the two phases, causing potential resource idleness or compute redundancy. Others split the prefill phase into chunks and fuse it with decode iteration, creating a dilemma between SLO compliance and high utilization. To address these issues, an efficient serving system should dynamically adapt compute allocation, decouple compute from memory management, and execute prefill and decode independently. We present MuxWise, an LLM serving framework that adopts a new paradigm, intra-GPU prefill-decode multiplexing, to meet these requirements. To fully exploit the paradigm, MuxWise integrates a bubble-less multiplex engine, a contention-tolerant estimator, and an SLO-aware dispatcher. Evaluation shows

that MuxWise improves peak throughput under SLO guarantees by an average of 2.20 $\times$ (up to 3.06 $\times$) over state-of-the-art baselines.

CCS Concepts: • Computer systems organization → Single instruction, multiple data; Cloud computing; • Software and its engineering → Process management.

Keywords: LLM Serving, PD-Multiplexing, Goodput

ACM Reference Format:

Yukang Chen, Weihao Cui, Han Zhao, Ziyi Xu, Xiaozhe Fan, Xusheng Chen, Yangjie Zhou, Shixuan Sun, Bingsheng He, and Quan Chen. 2026. Towards High-Goodput LLM Serving with Prefill-decode Multiplexing. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26)*, March 22–26, 2026, Pittsburgh, PA, USA. ACM, New York, NY, USA, 18 pages. <https://doi.org/10.1145/3779212.3790236>

1 Introduction

Large language models (LLM) services now perform well across diverse workloads [19, 30, 33]. At the request level, an LLM processes input in two phases: a prefill phase that produces the first token, followed by a decode phase that iteratively generates the remaining tokens. The ratio of input length (prefill) to output length (decode) varies across tasks [6, 43]. At the application level, tasks such as chatbot services or agent-based workloads [34] often consist of multiple turns of requests with shared context.

*Equal contribution.

[†]Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '26, Pittsburgh, PA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2359-9/2026/03

<https://doi.org/10.1145/3779212.3790236>

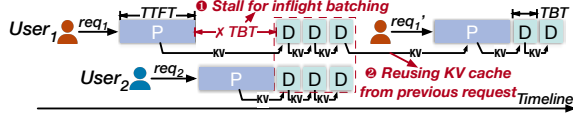


Figure 1. A typical workflow in LLM serving systems: User 1 sends two consecutive requests, with the second reusing the context from the first. User 2 sends 1 request.

To achieve high throughput for serving these workloads, existing LLM serving systems employ several optimizations. Figure 1 presents a typical workflow. While requests arrive at different times, inflight batching stalls the ongoing decode phase to prefill new requests and then processes all decode iterations together in a single batch. It greatly improves compute utilization for the memory-intensive decode phase [47]. Since multi-turn requests share context, LLM serving systems reuse intermediate results (i.e., the KV cache) both within and across requests through a KV cache pool [26, 52].

LLM services also impose stringent Service Level Objectives (SLOs). For instance, chatbot typically requires Time-To-First-Token (TTFT) under 500 ms for prefill and Time-Between-Tokens (TBT) under 100 ms for decode. Since prefill and decode interleave in an LLM serving system, SLO violations may arise. In Figure 1, inflight batching stalls ongoing decode. A long prefill can thus delay decode, potentially violating its SLO. To sustain high goodput—peak throughput with SLO guarantees—existing methods fall into two categories: disaggregated serving [32, 53] and chunked prefill [1].

As for disaggregated serving, Splitwise [32] separates the prefill and decode phases into distinct instances for SLO guarantees, which has two drawbacks. **Firstly, it cannot adapt to serving dynamics.** In Splitwise, GPUs are statically allocated at initialization. Under fluctuating request loads and diverse serving patterns, it often leads to resource underutilization. Secondly, **it decreases goodput due to shrinking the KV cache pool.** With the same number of GPUs, disaggregation allocates a separate cache pool for each instance, reducing the effective cache pool size. This lowers cache hit rate [45] (e.g., from 36.6% to 4.2%), leading to unnecessary recomputation and degraded goodput. Furthermore, while LoongServe [44] supports dynamic GPU allocation based on the request sequence length and execution phase, it cannot support the cross-request KV cache reuse, incurring significant recomputation overhead in multi-turn workloads.

Chunked-prefill [1] is another approach to meet decode SLOs. It splits the prefill phase into chunks within each GPU and fuses each chunk with a decode iteration. To ensure computational equivalence, each chunk reads the KV cache generated by all previous chunks. It ensures the decode SLOs by capping the token budget, defined as the sum of new tokens from the prefill chunk and the decode batch. By tuning

the chunk and decode batch sizes, it adapts to serving dynamics. Since it avoids disaggregation, it also prevents goodput loss from a reduced KV cache pool.

Unfortunately, chunking is not a free lunch. **It creates a dilemma between SLO compliance and high utilization.** Because prefill chunk and decode iteration must execute together, the token budget governs both decode SLO attainment and GPU saturation. Yet, finding a sweet budget in practice is infeasible. E.g., deploying a 70B LLM on 8 A100 GPUs requires a 4K budget to saturate the GPU, which is 8× larger than the SLO-compliant budget (256 for a 100 ms TBT SLO). Moreover, TBT in chunk-prefill is inflated by repetitive KV cache access from the prefill chunk. With extremely long reused context, common in multi-turn workloads, chunked-prefill may even fail to meet SLO guarantees (§2.3.2). **Ultimately, chunked-prefill cannot sustain high goodput.**

Achieving high-goodput LLM serving requires more flexible compute management. We propose intra-GPU prefill-decode (PD) multiplexing as a promising new serving paradigm. Specifically, the prefill and decode phases are executed on different streaming multiprocessors (SMs) within the GPUs. In the new paradigm, 1) compute partitions can be reconfigured with low overhead to adapt to serving dynamics; 2) multiplexed phases share GPU memory, keeping the KV cache pool efficient; 3) with spatial sharing, prefill and decode execute independently, avoiding the tradeoff between SLO compliance and utilization.

Realizing this paradigm is non-trivial. Firstly, phase coordination is still required to enable inflight batching and improve compute utilization. However, since prefill and decode latencies differ significantly, naive coordination often leaves GPU bubbles. Secondly, spatial multiplexing introduces unpredictable contention. Although existing approaches [10, 12, 13] partition compute, they provide little control over shared resources such as memory bandwidth.

To this end, we propose MuxWise, an LLM serving framework that achieves high goodput across diverse workloads. MuxWise comprises three modules: a *bubble-less multiplex engine*, a *contention-tolerant estimator*, and an *SLO-aware dispatcher*. The engine partitions prefill into layers with negligible overhead, aligning execution latencies for bubble-less multiplexing. The contention-tolerant estimator provides worst-case latency predictions by combining a solo-run predictor with a contention guard derived from one-time offline profiling. Built atop the engine and estimator, the SLO-aware dispatcher schedules diverse LLM requests efficiently by selecting multiplexing plans to maximize goodput.

We implement MuxWise on top of SGLang [52], extending it with PD multiplexing. MuxWise is evaluated extensively on both small and large LLMs using real-world workloads. Experiments show that MuxWise achieves an average 2.20× goodput improvement (up to 3.06×) over state-of-the-art solutions. In summary, our contributions are:

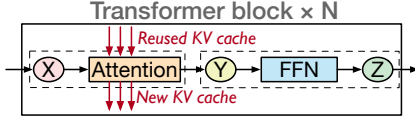


Figure 2. Main architecture of most LLMs.

Table 1. Diverse patterns of typical LLM tasks. Minimum, mean, and maximum values for each metric are reported. The input length includes the length of new and reused context.

	Input length	Output length	Reused length
ShareGPT [4]	4/226/1024	4/195/1838	\
LooGLE [25]	3380/30k/81k	2/15/326	\
OpenThoughts [17]	311/709/4633	684/8374/32k	243
Conversation [34]	891/7538/123k	1/342/2000	0/4496/120k
Tool&agent [34]	891/8596/123k	1/182/2000	0/4905/120k

- We identify key requirements for LLM serving with high goodput through a detailed analysis of prior works.
- We propose a new LLM serving paradigm—PD multiplexing—aligned with these requirements, and present a clean design to effectively serve LLMs with high goodput.
- We evaluate MUXWISE under diverse workloads, demonstrating its superiority over state-of-the-art solutions.

2 Background & Motivation

2.1 LLM Services

Architecture of LLMs. Most LLMs [5, 15, 37, 38] are built upon the transformer architecture [39], with model-specific modifications. Figure 2 illustrates a typical transformer layer, which is replicated multiple times to form an LLM model. Each transformer layer contains an attention layer and a feed-forward network (FFN) layer.

Attention computation requires access to all keys and values from processed tokens and also generates the keys and values of new tokens. To avoid redundant computation, LLM serving systems store this data in a KV cache. In the prefill phase, the KV cache is populated from the requests in previous turns. In each decode iteration, the KV cache is derived from earlier prefill and decode iterations.

Diverse workload patterns. Table 1 illustrates the diverse patterns of five typical LLM tasks. The first three are single-turn requests: ShareGPT [4] is a chatbot task, LooGLE [25] is a long-context understanding task, and OpenThoughts [17] is a reasoning task. LooGLE has a long input length due to long documents. Reasoning often requires long thought processes, so OpenThoughts tends to have a longer output length than others. Requests in OpenThoughts share the same system prompt, which is a constant input context (i.e., reused length in the table). Conversation and Tool&agent [34] are two real-world multi-turn tasks. The output tokens from earlier requests become the input context for later requests

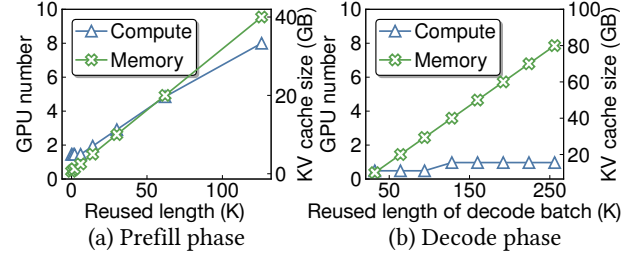


Figure 3. Required compute and memory for processing different phases under SLO constraints with varied reused context lengths. For prefill (a), the batch size is fixed at 1, the new context length is set to 2K, and TTFT is set to 400ms. For decode (b), the batch size is fixed at 32, and TBT is set to 100ms. These settings are commonly seen in online serving.

in the same session. We use these workloads to conduct experiments that both motivate and evaluate our design.

2.2 Characterization under SLO constraint

Many prior works [32, 44, 53] have investigated the relationship between resource requirements and SLO attainment concerning input length and batch size. Their experiments show that the prefill phase is compute-intensive, with compute demand growing linearly with input length, while the decode phase is memory-intensive. However, they mainly focus on the simple single-turn case, which does not consider the effect of reused input length.

Under these circumstances, we further study how the reused length impacts the compute and memory demands of prefill and decode. In our experiment, the reused length spans the range shown in Table 1, and LLaMA-70B [15] is deployed with tensor parallelism [51] on a server with 8 A100 GPUs. All GPUs are configured with the same partial compute resource, defined by the SM number. For each reused length, we determine the best-fit GPU partition ratio (denoted as GPU_{ratio}) to satisfy the SLO target. Figure 3 reports the total compute demand of LLaMA-70B under different reused lengths, computed as $GPU_{num} = GPU_{ratio} \times 8$.

As shown in Figure 3(a), prefill phase requires increasingly more compute resources to meet SLO targets as the reused length grows. In contrast, the compute demand of the decode phase shows less sensitivity. Thus, it is also critical to allocate more compute to the prefill phase as the reused length increases. Further, the distinct compute requirements of two phases necessitate a runtime compute resource partition for SLO attainment and high utilization.

Figure 3(b) shows that the KV cache required by both the prefill and decode easily reaches tens or even hundreds of gigabytes. This is common in multi-turn LLM services, which produce ultra-long reused contexts. It is preferable to keep the KV cache in the same memory space (aggregated serving) for efficient reuse across phases and requests.

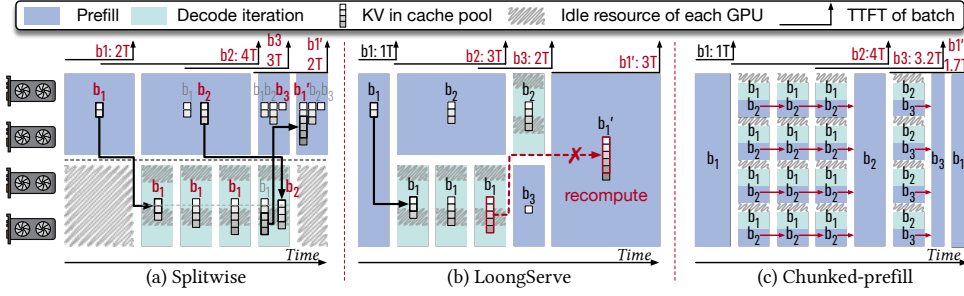


Figure 4. Processing four LLM request batches on 4 GPUs using (a) Splitwise, (b) LoongServe (c) chunked-prefill. All methods satisfy the TBT SLO (T per decode iteration). Specifically, b_1 arrives at $0T$, b_2 at $1T$, b_3 at $3T$, and b'_1 at $5T$. b'_1 denotes a subsequent request batch that reuses the KV cache of b_1 . Inefficient TTFTs are marked in red for each method. KV cache management is shown only for the two disaggregated methods, as they require migration or recomputation. In (a) and (b), solid black arrows represent migration, while dashed red arrows with cross markers denote recomputation. In (a), the KV cache column with a red b_i indicates the active batch. In (c), the red arrow denotes KV cache reads from earlier chunks.

In a nutshell, we make two observations: 1) *Appropriate and dynamic compute partition is essential for meeting the distinct SLO targets of different phases under diverse workloads.* 2) *Reusing the KV cache across phases and requests is critical for reducing redundant computation and improving goodput.*

2.3 Deficiencies of Existing Works

2.3.1 Disaggregated Serving. Disaggregating approaches partition GPUs across phases to meet the SLO targets in LLM serving and can be further divided into static and dynamic disaggregation methods. Figure 4-(a) illustrates the static approach (Splitwise [32]), while Figure 4-(b) shows the dynamic approach (LoongServe [44]).

Static disaggregation. As shown in Figure 4-(a), there is a prefill instance and a decode instance with Splitwise [32]. Each instance occupies two GPUs statically and has its own KV cache pool. The GPU number is static after the instance is initialized. In this case, Splitwise suffers from two problems.

First, Splitwise does not adapt to serving dynamics. For example, when batch b_1 arrives, only two GPUs process the prefill while the other two GPUs for decoding remain idle. In online serving, such idle periods are common as request loads fluctuate. *Second, the coupled management of compute and memory introduces further inefficiencies.* For instance, if the decode phase of b_1 in Figure 4-(a) requires two GPUs to store the KV cache, the system must also allocate two GPUs for computation. Since compute and memory requirements are misaligned, as shown in Figure 3-(b), the GPUs' compute resources may be underutilized.

In addition, each instance must maintain its own model weights and KV cache pool. As a result, the KV cache pool in Figure 4-(a) is at most half the size of that with four GPUs under non-disaggregated execution. Furthermore, experimental results in Figure 5 show that this reduced capacity

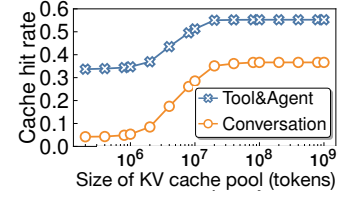


Figure 5. Cache hit rates under varying capacities of the KV cache pool. The eviction policy is Least Recently Used. For serving a 70B LLM, achieving the optimal hit rate requires 3.3 TB of memory. Workload trace details are shown in Table 1.

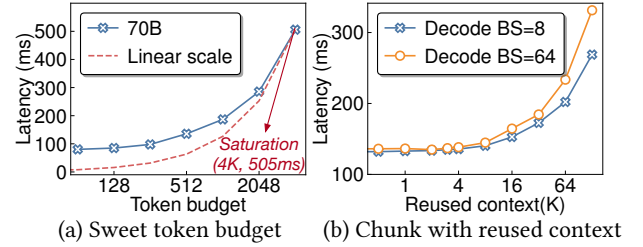


Figure 6. (a) Sweet spot of the token budget in chunk-prefill. The decode uses a fixed batch size of 32, with each request having a reused context length of 1K tokens. (b) Latencies with varied reused context of the fused prefill chunk in chunk-prefill. The token budget is fixed at 512, and the reused context length of decode phase is the same as in (a).

sharply lowers the KV cache hit rate in multi-turn workloads, ultimately degrading the system's goodput.

Dynamic disaggregation. LoongServe [44] supports dynamic GPU partitioning across the two phases. Specifically, it scales GPU resources based on the sequence length and execution phase. As shown in Figure 4-(b), when batch b_1 arrives, the scheduler assigns four GPUs to prefill. After prefill, it scales down to two GPUs for the decode iterations.

However, LoongServe still causes idleness due to coupled management, and worse, it trades KV cache reuse for adaptiveness needed in serving dynamics. To avoid duplication, it immediately releases the KV cache on original GPUs. Thus, KV caches are reused only from prefill to decode within a single request and cannot be reused across multi-turn requests. In Figure 4-(b), when b'_1 needs to reuse the KV cache generated by b_1 , LoongServe recomputes the entire KV cache.

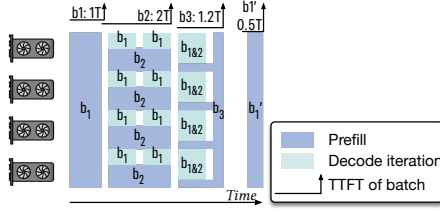


Figure 7. An ideal solution: prefill-decode multiplexing.

2.3.2 Chunked-prefill. Chunked-prefill [1] adopts intra-GPU compute fusion. As shown in Figure 4-(c), it splits prefill into chunks and fuses each chunk with a decode iteration. To guarantee decode SLOs, chunked-prefill caps the token budget, which is the sum of new tokens from the prefill chunk and the decode batch. While chunked-prefill has known drawbacks such as quadratic memory overhead [53], we find another drawback. Specifically, chunking introduces a dilemma between SLO attainment and utilization.

Figure 6-(a) presents TBT in Chunked-prefill of varying token budget. In this experiment, the decode iteration for fusion has a static batch size of 32 and a reused context length of 1K tokens, and Llama3-70B is deployed on a server with 8 A100 GPUs. As shown, the latency does not increase linearly with the token budget until it reaches 4K. This indicates that saturating the GPUs requires a prefill chunk with input length of $(4K - 32)$. However, the corresponding latency is 505ms, far above the typical TBT SLO target ($< 100ms$).

Figure 6-(b) presents TBT in Chunked-prefill with varying reused context lengths of the prefill. In this experiment, the token budget is fixed at 512, and the reused context length of decode iteration is 1K. As shown, TBT increases noticeably after the reused context exceeds 4K. This reused context length is common in long-context understanding and multi-turn workloads, as shown in Table 1. In such cases, Chunked-prefill easily leads to SLO violations.

2.4 New Paradigm & Challenges

As shown in Figure 7, we propose an intra-GPU prefill-decode (PD) multiplexing paradigm to overcome the above limitations. Specifically, prefill and decode dynamically share the compute resources (SMs) within each GPU. By reserving sufficient SMs to satisfy decode SLOs and assigning the remaining SMs to prefill, high-goodput LLM serving is achieved. PD multiplexing overcomes the limitations of prior methods, benefiting from the following abilities.

First, multiplexing enables dynamic and adaptive compute management. As shown, compute resources can be flexibly allocated between the two phases to maximize system goodput while guaranteeing SLOs. Second, multiplexing decouples compute from memory management. Although the two phases partition compute resources, they share the memory space on each GPU, enabling efficient KV cache reuse. Third, multiplexing allows prefill and decode to run independently

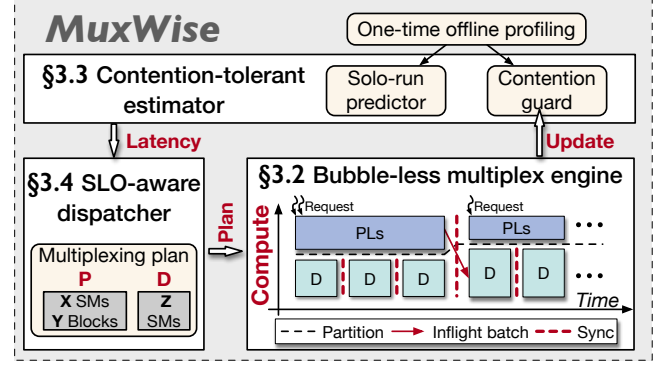


Figure 8. Architecture overview of MuxWise.

without stalling one another, avoiding the dilemma between SLO attainment and system goodput.

However, integrating intra-GPU multiplexing into existing LLM serving systems is non-trivial. There are two challenges to realizing this paradigm. **C-1: GPU bubbles from naive integration.** Current systems have frequent prefill-decode interactions due to the inflight batching mechanism. One phase can easily block the other, creating GPU bubbles. **C-2: Unmanaged contention in spatial multiplexing.** Existing techniques [10, 12, 13] partition only SMs while leaving memory bandwidth unmanaged. As a result, memory bandwidth contention can lead to SLO violations.

3 MuxWise's Design

3.1 Architecture Overview

Based on the PD multiplexing paradigm, we propose MUXWise, an LLM serving framework that achieves high goodput on diverse workloads. Figure 8 shows the overview of MUXWise that comprises (1) a bubble-less multiplex engine, (2) a contention-tolerant estimator, and (3) an SLO-aware dispatcher.

To enable PD multiplexing with bubble-less coordination, the engine partitions prefill into layer-wise execution, aligning its latency with decode execution. Notably, layer-wise execution incurs negligible overhead and avoids the inefficiencies of chunk-prefill. In addition, it provides an extra benefit: preempting ultra-long prefills to prevent the SLO violations caused by them.

To avoid SLO violations caused by unpredictable contention, the estimator provides worst-case latency estimates by combining a solo-run latency predictor with a contention guard. Both components are built from one-time offline profiling for each LLM on a given hardware. They consider five key factors: reused length, input length, output length, decoding batch size, and partition configuration. LLM-specific factors are extracted from workload traces to guide profiling.

As requests arrive, the SLO-aware dispatcher leverages the engine and estimator to schedule prefill layers and decode iterations dynamically for high goodput. Specifically, the

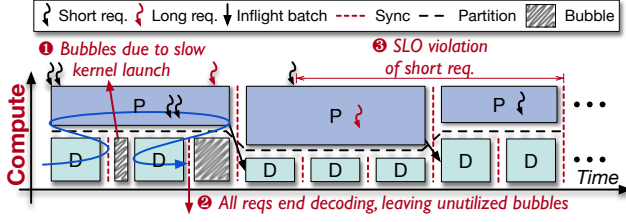


Figure 9. Bubbles and SLO violation of naive intra-process multiplexing. $\rightarrow$ represents the kernel launch order.

dispatcher reserves best-fit SMs to satisfy decode SLOs based on the worst-case estimation, and assigns the remaining SMs to prefill. Meantime, during online serving, it further refines the contention guard using runtime execution data.

3.2 Bubble-less Multiplex Engine

3.2.1 Spatial Multiplexing Technique. According to the analysis in §2.4, MuxWise imposes two requirements for PD multiplexing. First, the compute resources must be dynamically partitioned between the two phases with low overhead. Second, the memory space must be shared across the phases to enable efficient KV cache reuse. To meet these requirements, we examine existing approaches for spatially partitioning GPU compute across tasks.

We categorize these approaches into two types: inter- and intra-process partitioning. Inter-process approaches, such as CUDA MIG [12] and CUDA MPS [13], cannot provide flexible compute resource adjustment, let alone the introduced cross-process communication between prefill and decode. In contrast, the intra-process approach GreenContext [10] enables low-overhead resource adjustment by binding CUDA streams to specific SMs, with reconfiguration costing only a stream synchronization (on the order of microseconds). Furthermore, because both phases reside in the same process under GreenContext, they can directly share the same memory space for maintaining a single KV cache pool.

3.2.2 Inefficiencies from Naive Integration. With intra-process compute partitioning, a naive way to support PD multiplexing is to launch the ongoing decode iteration before the prefill phase of new request. This ordering is motivated by launch latency difference: launching a decode iteration takes less than 0.5 ms, whereas launching a prefill phase takes tens of milliseconds.

Ideally, the launch latency of either a prefill phase or decode iteration can be reduced to a single CUDA graph launch ($\sim 0.5\text{ms}$). However, CUDA graph requires offline construction with static configuration and incurs memory overhead. In prefill phase, both batch size and input length vary, whereas decode iteration varies only in batch size. Thus, single-graph optimization is feasible only for decode phase with several selected batch sizes [40], while applying it to prefill phase

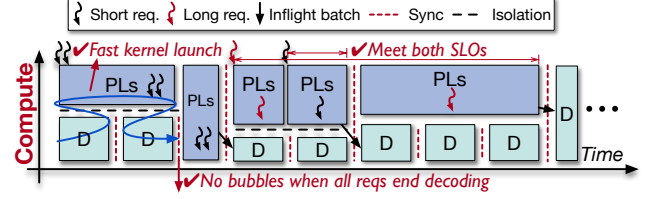


Figure 10. Bubble-less coordination using layer-wise scheduling for the prefill phase and graph-level scheduling for the decode phase. PLs is the short for prefill layers.

would require capturing much more graphs, incurring unacceptable memory overhead.

Prefill phase can be optimized through piecewise CUDA graph [40], which splits the prefill phase into multiple layer-wise CUDA graphs. It still incurs $\sim 10\text{ ms}$ launch overhead for Llama-70B on 8 A100 GPUs. Fortunately, prefill phases consists of long-duration kernels, which are typically longer than the launch time. It does not suffer noticeable performance degradation from launch overhead in most cases.

To this end, when both a prefill phase and a decode iteration are pending, MuxWise prioritizes launching the decode iteration; otherwise, the SMs allocated for the decode phase would remain idle for tens of milliseconds. However, this naive approach still introduces two types of GPU bubbles and can also lead to SLO violations.

Firstly, when the prefill launch time exceeds the execution time of a decode iteration, next decode iteration cannot launch in time, and GPU bubbles occur. As shown on the left of Figure 9, a bubble appears between two decode iterations because the serving system must return newly generated tokens after each iteration.

Secondly, bubbles can arise from unpredictable termination of decode. As depicted in the middle of Figure 9, all requests in a decode batch may finish token generation while a concurrent prefill phase has already been launched. Due to the non-preemptive nature of GPU execution, the launched prefill cannot be interrupted to reclaim compute resources.

Thirdly, SLO violations may occur due to workload skew among requests, as illustrated in the upper-right of Figure 9. Context lengths can vary significantly, with short conversations coexisting alongside long-text summarizations. In such cases, a short request may suffer long queuing delays while waiting for the prefill of an ultra-long request. If the short request has limited SLO slack, it is likely to miss its deadline.

3.2.3 Bubble-less Coordination. The above inefficiencies stem from the large latency discrepancy between the prefill and decode phases. This is because prefill phases typically take longer to launch and execute, and the execution time of both phases is highly variable at runtime. To address this, we propose layer-wise execution for prefill and query-based synchronization.

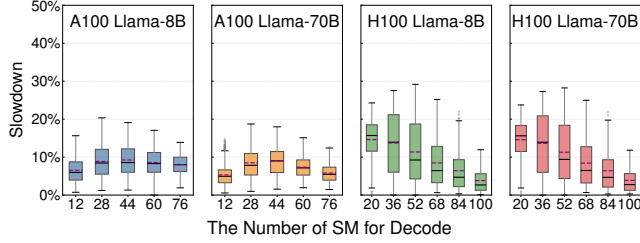


Figure 11. Slowdowns in decode due to contention with different multiplexing configurations of prefill and decode across models and GPUs.

Layer-wise execution for prefill. As shown in Figure 10, MuxWise splits the prefill phase into layers (PLs). Based on this new granularity, MuxWise eliminates GPU bubbles and prevents SLO violations. For the first type of bubble, MuxWise can launch enough PBs to occupy compute resources for prefill, and return in time before the decode phase finishes computation. For the second type, MuxWise switches the execution of later prefill layers into a new GreenContext, just after the decode phase terminates. For SLO violations caused by long requests, layer-wise execution enables preemption, allowing short requests to be prioritized, thereby meeting the SLO targets of both. Importantly, layer-wise execution incurs negligible overhead, since LLMs are inherently structured as multiple transformer layers.

Query-based synchronization. In addition to the above bubbles, inflight batching can also introduce GPU bubbles. Specifically, when the last prefill layer completes, it must block the next decode iteration to merge requests into the decode batch. This blocking creates small bubbles, since prefill and decode rarely finish simultaneously. To address this, MuxWise employs query-based synchronization that periodically polls CUDA events. MuxWise continues launching decode batches and prefill layers asynchronously, and when an event is observed complete, the corresponding prefill request is immediately merged into the current decode batch.

3.3 Contention-tolerant Estimator

When the prefill and decode phases are spatially multiplexed, contention can arise, particularly from unmanaged resources such as memory bandwidth. We begin by analyzing contention between the two phases under spatial multiplexing and then introduce our modeling method.

3.3.1 Contention Analysis. While GreenContext supports precise compute resource allocation, it cannot manage memory or network bandwidth. In particular, efficient techniques for bandwidth management are lacking. Worse, current GPUs do not expose runtime monitoring of bandwidth usage, and both prefill and decode phases can heavily consume bandwidth, making contention hard to predict.

Table 2. Compute analysis for prefill and decode phases.

	Attention	FFN
Prefill w/o cache	$O(Ld^2 + L^2d)$	$O(Ld^2)$
Prefill w/ cache	$O(nd^2 + Lnd)$	$O(nd^2)$
Decode	$O(d^2 + (r + 1)d)$	$O(d^2)$

To evaluate the impact on execution slowdown, we extensively profile prefill and decode under multiplexing using Llama-8B and Llama-70B. Figure 11 reports the decode slowdown on servers with 8 A100 and 8 H100 GPUs. The x-axis denotes the number of SMs allocated to the decode batch, with the remaining SMs assigned to the prefill batch. For each configuration, the prefill batch’s total context length (reused + new) ranges from 1,024 to 128K tokens, whereas the decode batch’s reused length ranges from 1,024 to 1,024K tokens. This profiling takes one week.

As shown in Figure 11, contention-induced slowdown ranges from nearly zero to about 30% across different partition configurations and GPUs. The high variation across hardware partitions indicates the inherent unpredictability of contention slowdown. Although both models exhibit similar slowdown trends on the same GPUs due to their architectural similarity, this observation does not aid contention modeling. Meanwhile, results for prefill are similar but omitted due to space constraints.

3.3.2 Worst-case Estimation for SLO guarantee. To mitigate the risk of SLO violations caused by unpredictable contention in online serving, MuxWise introduces a worst-case latency estimation method tailored for SLO guarantees. *The key observation is that precise latency prediction is not the only way to guarantee SLO. What matters is ensuring that the latency of a scheduled phase, given its allocated compute resources, does not exceed the predefined target.* Thus, MuxWise performs worst-case estimation by first predicting its solo-run latency, and then applying a maximum slowdown factor.

Solo-run predictor. To predict solo-run latency, we analyze the compute complexity of prefill and decode, and construct a predictor using offline profiling. The latency of each prefill or decode iteration is determined by the token lengths of the reused and new context. Table 2 summarizes the compute analysis of the prefill and decode phases with a batch size of 1. The key factors are as follows:

- d : The hidden dimension of each token’s representation.
- L : The total token length.
- r : The token length of the reused (cached) context.
- $n = L - r$: The token length of the new context.

Based on the complexity analysis in Table 2, we build latency prediction models for the prefill and decode phases. The prefill model is given in Equation 1, and the decode model in Equation 2, where all θ terms are coefficients. We

separate the models because state-of-the-art serving frameworks adopt different execution paths for these two phases, including distinct GPU kernel implementations and launch methods.

$$T_{Prefill} = \theta_1 \cdot \sum_i^{bs} n_i^2 + \theta_2 \cdot \sum_i^{bs} n_i \cdot r_i + \theta_3 \cdot \sum_i^{bs} n_i + \theta_4 \quad (1)$$

$$T_{Decode} = \theta_1 \cdot \sum_i^{bs} r_i + \theta_2 \cdot bs + \theta_3 \quad (2)$$

The trained models achieve high accuracy, with a maximum deviation of 8.16% for prefill and 8.84% for decode, effectively supporting MuxWise’s online scheduling. Meanwhile, the offline profiling for training the solo-run predictor can be completed within a few hours, which is acceptable. It is a one-time effort per LLM-machine pair and has been widely adopted in prior LLM serving work [1, 44, 53].

Contention guard. To provide the maximum slowdown factor, MuxWise introduces the contention guard. Specifically, the contention guard provides slowdown factors only for decode, which will be explained in §3.4.1. The contention guard is built using data collected through grid-sampling-based profiling. This profiling spans five variables: the number of new and reused tokens in prefill, the batch size, and total reused tokens in decode, and the partition configuration. For each pair of prefill and decode iterations to be estimated, the contention guard returns the maximum slowdown factor of the grid cell they fall into as the estimation result.

Building such a contention guard incurs much higher offline profiling overhead than the solo-run predictor, since pairwise profiling is needed to obtain slowdown data. Fortunately, extensive profiling shows that slowdown remains within a limited range, with a maximum of 20% on A100 GPUs and 30% on H100 GPUs. This indicates that even with coarse-grained profiling, worst-case latency inflation does not exceed 30%.

To this end, we initialize the contention guard using coarse-grained grid sampling. Specifically, we sample variables such as new and reused tokens in prefill, as well as single-request reused tokens in decode batches, at powers-of-4 granularity, ranging from 2K to 128K. The sampled decode batch sizes follow SOTA serving frameworks (around 20 batch sizes). We partition GPUs at the granularity of 16 SMs, yielding 6 configurations for A100 and 7 for H100. In total, the number of samples per LLM-machine pair is calculated as $(4 \times 4 - 1) \times 4 \times 20 \times 6 \approx 7K$, which can be collected within 12 hours. In the equation, we exclude the case with 128K new and 128K reused tokens in prefill, since 128K is the maximum context window supported by mainstream LLMs [5, 15].

The reason for using 16 SMs as the granularity are twofold: (1) kernels on H100 and newer GPUs requires 16 SMs, due to using new features like thread block cluster [3], and (2)

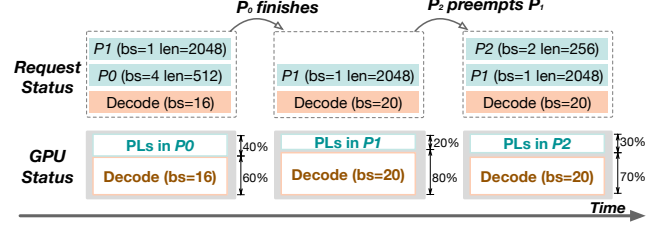


Figure 12. The dispatching policy of MuxWise.

experiments indicate that 16 SMs already deliver strong performance improvements. Finer-grained scheduling offers little benefit while increasing memory overhead.

Furthermore, MuxWise leverages runtime execution data to continuously update the contention guard, thus refining its SLO guarantees. Even with the coarse-grained contention guard, MuxWise already outperforms existing baselines in both SLO attainment and system goodput (§4).

3.4 SLO-aware Dispatcher

With bubble-less multiplexing and contention-tolerant modeling, we introduce MuxWise’s detailed dispatching policy.

3.4.1 Priorities of prefill and decode. In this work, we focus on the scheduling within a single serving instance. In MuxWise, we prioritize SLO attainment for the decode phase and process the prefill phase as early as possible. SLO attainment for the prefill phase is not directly guaranteed for two reasons. Firstly, although we prioritize the decode phase, we only allocate just-enough compute resources for it. Since the remaining compute resources are allocated to the prefill phase, its SLO is generally expected to be met. Secondly, when SLO violations occur for the prefill phase, it indicates that the inference load has exceeded the peak capacity of the current LLM serving instance. In such cases, further scheduling efforts would no longer improve performance.

This is also why the contention guard in the contention-tolerant estimator only provides a maximum slowdown factor for decode. When predicting the prefill phase, MuxWise does not need an accurate or worst-case estimate. It only requires that the predicted latency of the launched prefill layers exceeds that of the corresponding decode iteration, ensuring full utilization of the allocated compute resources.

3.4.2 Dispatching policy. Building on the above analysis, Figure 12 illustrates MuxWise’s SLO-aware dispatching policy. The system makes scheduling decisions after each prefill batch completes and at the end of each decode iteration. Specifically, the dispatcher selects prefill layers either from the ongoing prefill batch or from a new batch in the request queue, and allocates compute resources between the prefill and decode phases.

As shown in Figure 12, the dispatcher allocates a best-fit number of SMs (60%) to decode and assigns the remaining

SMs (40%) to prefill phase P_0 . The resource partition satisfies the decode SLO and maximizes prefill throughput guided by the contention-tolerant estimator. To support layer-wise execution in the multiplex engine, the estimator computes the number of prefill layers to launch as $N_{PL} = \lceil (T_d \times N_T) / T_P \rceil$, where T_d is the estimated decode latency, T_P is the estimated prefill latency, and N_T is the number of transformer layers in the served LLM.

Once P_0 finishes computation, it is merged into the decode batch, increasing the batch size to 20. The scheduler then retrieves a new prefill batch P_1 and adjusts the partition to 20% SMs for prefill and 80% for decode to meet SLO targets.

Later, when a new prefill batch P_2 arrives, it would normally wait for P_1 to complete. However, because P_1 has a long input length, this delay risks violating P_2 's SLO. To avoid this, MuxWise allows P_2 to preempt P_1 , provided that preemption does not cause P_1 to miss its own TTFT SLO.

MuxWise does not allow recursive preemption. For example, after P_2 preempts prefill batch P_1 , no other batch may preempt P_2 . This design is reasonable, as short requests typically preempt long ones, and preempting a short request in turn would likely cause it to miss its SLO. MuxWise checks SLO attainment only when a prefill batch is preempted; otherwise, it prioritizes processing the active prefill batch as quickly as possible. Notably, preemption in MuxWise is optional. Even when disabled, MuxWise still delivers substantial performance improvements over the baselines.

4 Evaluation

4.1 Experimental Setup

Testbed. We mainly evaluate MuxWise on a server equipped with 8 A100-80GB GPUs. The GPUs are interconnected via NVLINK, providing 600 GB/s of bandwidth. We also evaluate MuxWise on two additional servers to demonstrate its effectiveness on newer GPUs and larger LLMs: one with 8 H100-SMX5-80GB GPUs and another with 8 H200-SMX5-141GB GPUs. These servers offer higher compute capability and larger GPU memory. All experiments are conducted with PyTorch 2.6.0 [31]. MuxWise is implemented using SGLang [52] version 0.4.10post2. The GPU driver version is 570.124.06, and the CUDA version is 12.8.

Models. We primarily evaluate MuxWise using two LLMs from the Llama family [15, 37, 38]: Llama-8B and Llama-70B. These models differ in size and represent the most commonly hosted LLMs in the cloud. We also evaluate a larger MoE model, Qwen3-235B with 22B activated, to demonstrate MuxWise's generality.

Baselines. We compare MuxWise against 3 state-of-the-art solutions for efficient LLM serving. Model parallelism techniques such as tensor parallelism [51] are employed to parallelize the deployed models. For MuxWise, we fix the tensor parallelism degree to 8. Details of each baseline's model parallelism configuration are provided when the baseline is

introduced. For all systems, the KV cache memory pool is configured as large as possible to maximize throughput.

- **Chunked-prefill in SGLang [1]:** This version of SGLang is equipped with chunked-prefill, as proposed by SARATHI-Serve [1]. We follow SARATHI-Serve's methodology to calculate the token budget for each workload prior to experiments. It is offline tuned under specific TBT targets for each model. Unlike SARATHI-Serve, which serially executes prefill and decode attention kernels, SGLang leverages Flashinfer [46], a high-performance inference kernel library, to fuse them into a single kernel. It is expected to deliver performance similar to POD-attention [21].
- **NanoFlow [54]:** This is an enhanced version of chunked-prefill with operator-level intra-GPU multiplexing, targeting near-optimal throughput under a relatively loose SLO requirement (200 ms). It requires a large token budget (at least 1024) to achieve this goal. However, such a token budget cannot meet the SLO requirements of modern LLM serving (≤ 100 ms). We use the same token budget as chunked-prefill for NanoFlow.
- **LoongServe [44]:** This is a dynamic disaggregated serving system. We adopt its model-parallelism configuration. For Llama-70B, sequence parallelism is set to 2 and tensor parallelism to 4. For Llama-8B, sequence parallelism is set to 4 and tensor parallelism to 2. It does not support new LLMs like MoE models.
- **Disaggregated serving in SGLang (SGLang-PD in short):** This is the latest implementation of static disaggregation with KV-cache sharing across phases and requests. The P:D ratio is 1:1, with tensor parallelism set to 4 for each instance. DistServe [53] does not support KV-cache sharing across requests, making it unsuitable for modern LLM services. We also evaluated Dynamo [14], which performed substantially worse than SGLang-PD. Therefore, we use SGLang-PD as the state-of-the-art baseline of static disaggregation for evaluation.

Metrics. MuxWise targets goodput improvement. So, following prior works [1, 32, 34, 53], we use tail latency (e.g., P99) to assess SLO attainment. Meanwhile, there is also another metric to measure the SLO guarantee during decode phase: TPOT (timer per output token). In comparison, TBT accounts the latency of each individual token, whereas TPOT is an average metric that may mask the poor performance of some tokens [42]. Thus, we choose TBT over TPOT for a stricter SLO metric. We set the TBT SLO target to 50ms for Llama3-8B and 100ms for Llama3-70B, following prior works [1, 34]. We regard MuxWise's ability to deliver better TTFTs under skewed workloads as an additional benefit of the new serving paradigm. Moreover, it breaks the first-come-first-serve model used in other baselines. Thus, we only evaluate TTFT per token in §4.4.3.

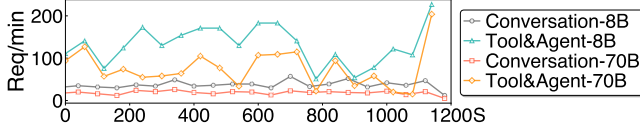


Figure 13. The two real-world workload traces after scaling.

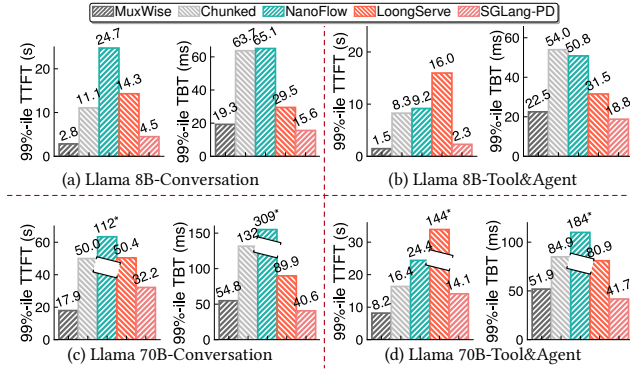


Figure 14. 99%-ile TTFT and TBT for Llama-8B and Llama-70B on real-world Conversation and Tool&Agent workloads. Chunked represents chunked-prefill in SGLang. Values marked with * are too large; we clip their corresponding bars, so the bar height only decodes their relative size.

4.2 Evaluation on Real-world Workloads

4.2.1 End-to-end Performance. We begin by evaluating MuxWise with Llama-8B and Llama-70B under real-world workload traces. Figure 13 shows their request rates after scaling down, as they are originally from a large cluster. As illustrated, they show bursty request patterns (up to 13× spike within 1min).

Figure 14 shows the latency distribution of TTFT and TBT. Although the real-world traces are scaled down to a modest level, some baselines still easily reaches its peak throughput and enters an unstable state (NanoFlow in Figure 14-(c) and LoongServe in Figure 14-(d)). After omitting unstable results, MuxWise achieves average 99%-ile TTFT speedups of 3.57×, 5.98×, 4.65×, and 1.66× over chunked-prefill, NanoFlow, LoongServe, and SGLang-PD, respectively. MuxWise and the two disaggregated solutions consistently meet the TBT SLO, whereas chunked-prefill and NanoFlow fails in most cases. SGLang-PD achieves shorter TBT than MuxWise, as it statically reserves more compute resources for the decode instance.

Compared to chunked-prefill, MuxWise avoids the dilemma between SLO compliance and high utilization, bringing better performance for both prefill and decode phases. While tuning the token budget, we observe that either increasing or reducing it fails for SLO guarantee. This is because the reused length in prefill phase in the two workloads can reach up to 50K tokens. Further splitting the prefill into smaller chunks

Table 3. Results of other metrics for Llama-70B on Conversation workloads in Figure 14-(c).

	TTFT (s)		TBT (ms)		E2E (s)		TPOT (ms)	
	Avg.	P50	Avg.	P50	Avg.	P50	Avg.	P50
MuxWise	3.1	1.4	30.2	25.0	13.1	12.2	31.1	28.3
Chunked	12.0	7.2	45.3	49.3	27.0	23.3	46.9	45.7
NanoFlow	51.4	52.3	105.6	98.9	83.4	84.2	120.2	103.2
LoongServe	17.7	14.7	61.0	58.3	38.4	35.8	62.3	60.6
SGLang-PD	7.38	3.95	32.9	32.5	18.4	16.4	33.5	33.2

does not help control the TBT. MuxWise’s PD multiplexing avoids this issue entirely.

NanoFlow performs worse than the original chunked-prefill. This is because, built atop chunked-prefill, NanoFlow is designed to overlap compute-bound kernels with memory-bound or communication kernels. To achieve this, it requires a large token budget (1024 in its paper) to ensure that chunked-prefill as a whole remains compute-bound. However, in Figure 14, the token budget has to be reduced to 256 to meet TBT SLO targets, where chunked-prefill is no longer compute-bound. The long reused context length in the two evaluated real-world traces further makes chunked-prefill harder to be compute-bound. Thus, NanoFlow degrades due to overlapping memory-bound kernels.

The situation worsen for NanoFlow with Llama-70B in Figure 14-(c&d). This could be attributed to the inherent model weight reload of intra-GPU overlapping. NanoFlow split each chunk into 2 nano batches, thus duplicating loading for each decode iteration [54]. When evaluating with Llama-70B, the reloading overhead is amplified due to the larger model size. Conversely, MuxWise duplicates loading only once during prefill, which typically co-runs with tens of decode iterations. Because prefill is compute-intensive and the reload is amortized over the entire phase, MuxWise imposes negligible bandwidth pressure, which is marginal relative to its overall benefits. While NanoFlow performs poorly on the two real-world workloads, it outperforms chunked-prefill for short input sequences without cross-request context length reuse (§4.3).

Against the two disaggregated solutions, MuxWise achieves significantly better TTFT. In LoongServe, instance scaling releases the KV cache needed for reuse in the prefill phase, causing redundant recomputation. In SGLang-PD, static disaggregation often leaves decode instances idle under fluctuating real-world workloads. In contrast, MuxWise avoids KV migration and adapts to dynamic workloads through intra-GPU compute partition reconfiguration.

4.2.2 Other Latency Metrics. In this experiment, we report results for other metrics, such as end-to-end latency and TPOT. We also present these results using other statistical measures, including average and P50 values. Table 3 shows the results on the Conversation workload with Llama-70B,

Table 4. Results of other metrics for Llama-70B on Tool&Agent workloads in Figure 14-(d).

	TTFT (s)		TBT (ms)		E2E (s)		TPOT (ms)	
	Avg.	P50	Avg.	P50	Avg.	P50	Avg.	P50
MuxWise	1.3	1.0	27.2	24.1	4.0	1.6	33.1	27.7
Chunked	2.4	1.1	30.5	21.2	5.5	2.3	45.9	47.1
NanoFlow	2.8	1.2	58.8	42.4	7.4	2.5	70.3	62.6
LoongServe	59.9	56.0	52.4	50.8	65.2	61.0	56.2	54.6
SGLang-PD	2.1	1.5	31.6	31.0	5.2	2.3	37.1	33.7

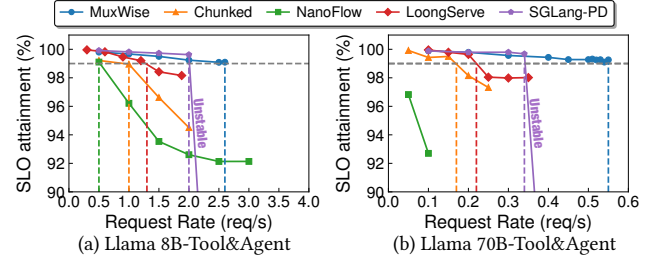
while Table 4 shows the results on the Tool&Agent workload with Llama-70B in §4.2.1. Results in other settings are similar and are omitted due to space constraints.

As shown in the two tables, while MuxWise focuses on improving high goodput, it also consistently outperforms the baselines across the reported metrics. There is only one outlier in the P50 TBT of Table 4, and the values are very close. This can occur because the P50 TBT in chunked-prefill may correspond to the latency of a pure decode iteration.

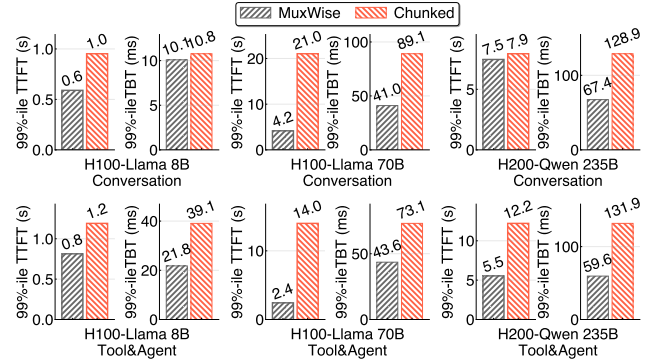
4.2.3 SLO Attainment and Goodput. We also measure the SLO attainment of TBT and the corresponding goodput to evaluate the effectiveness of MuxWise in meeting SLO compliance. In this experiment, we extract requests from the Tool&Agent trace but replace their arrival timestamps with those generated by a Poisson process at varying rates, following prior work [44]. We stop testing once the serving system becomes unstable or fails to meet the TBT SLO target.

Figure 15 shows the SLO attainment results under gradually increasing workloads. Under the constraint of meeting the 99%-ile SLO guarantees, MuxWise achieves 2.6×, 5.2×, 2.0×, and 1.3× higher goodput than chunked-prefill, NanoFlow, LoongServe, and SGLang-PD, respectively for Llama-8B; and 3.06×, 2.62×, and 1.62× higher than chunked-prefill, LoongServe, and SGLang-PD for Llama-70B. NanoFlow never meets the SLO even with a small chunk size of 64 for Llama-70B; therefore, the corresponding goodput improvement is omitted. Table 5 further shows the corresponding token throughput and GPU utilization of MuxWise and baselines. GPU utilization is an aggregated metric reported by NVIDIA Nsight Systems, that reflects the fraction of active SMs as well as the utilization of intra-SM resources.

Chunked-prefill, and NanoFlow fails to meet the TBT SLO even at lower request rates than the other two baselines. This is because chunking is largely ineffective at reducing TBT in real-world LLM services, where cross-request interactions are common. Compared to LoongServe, MuxWise achieves higher goodput by avoiding recomputation in multi-turn requests. Compared to SGLang-PD, MuxWise achieves higher goodput through a larger KV-cache pool and reduced idleness caused by static disaggregation. Meanwhile, MuxWise achieves shorter TTFT across all cases (up to 9.16×).

**Figure 15.** SLO attainment for Llama-8B and Llama-70B on Tool&Agent workload with varied request rates.**Table 5.** Token throughput and GPU utilization for Llama-8B and Llama-70B on Tool&Agent workload under Goodput.

Model	Llama-8B		Llama-70B	
Metrics	Token/s	GPU Util.	Token/s	GPU Util.
MuxWise	25397	88.1	7430	84.0
Chunked	9768	63.8	2269	66.1
NanoFlow	4884	55.1	—	—
LoongServe	12698	75.3	2936	70.1
SGLang-PD	19535	P(72.4)/D(83.4)	4538	P(67.1)/D(81.9)

**Figure 16.** 99%-ile TTFT and TBT for Llama-8B and Llama-70B on a server with 8 H100 GPUs and 99%-ile TTFT and TBT for Qwen-235B on a server with 8 H200 GPUs.

4.2.4 More Advanced GPUs and Larger LLM. To demonstrate MuxWise’s effectiveness on other GPUs and LLMs, we evaluate it with Llama-8B and Llama-70B on a server with 8 H100 GPUs, and with Qwen-235B on a server with H200 GPUs. In this experiment, we only compare MuxWise with chunked prefill. LoongServe does not support new MoE models like Qwen-235B, and disaggregated serving solutions are also infeasible for Qwen-235B, even though each H200 has 141 GB of GPU memory. Figure 16 shows the experimental results. Across all cases, MuxWise achieves an average 2.28× speedup on 99%-ile TTFT and an average 1.81× speedup on 99%-ile TBT. These consistent improvements demonstrate the generality of MuxWise’s serving paradigm across diverse hardware and larger, newer LLMs.

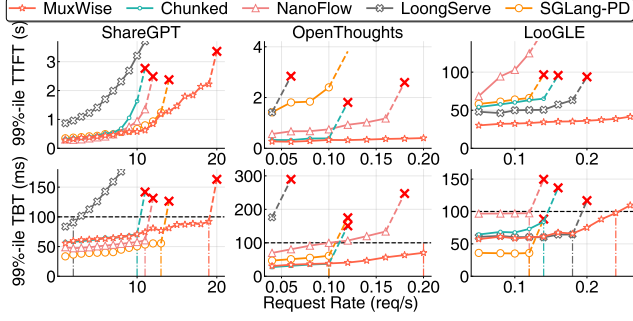


Figure 17. 99%-ile TTFT and TBT with Llama-70B on three types of synthetic workloads.

4.3 Evaluation on Diverse Synthetic Workloads

To better demonstrate MUXWise’s effectiveness, we further evaluate it under three synthetic workloads. In the rest of the evaluation, we focus on Llama-70B due to space constraints, as results on other models are similar. Requests are generated by sampling inputs from ShareGPT [4], Openthoughts [17], and LooGLE [25], with arrival rates gradually increased following a Poisson process. Among these, only Openthoughts requests share a short system prompt. We select these workloads because they represent three typical patterns: moderate input and output, short input with ultra-long output, and ultra-long input with short output.

Figure 17 shows 99%-ile TTFT and TBT of MUXWise and three baselines. On ShareGPT, MUXWise achieves goodput improvements of 1.9 $\times$, 1.73 $\times$, 9.5 $\times$, 1.46 $\times$ over chunked-prefill, NanoFlow, LoongServe, and SGLang-PD, respectively. On LooGLE, it achieves 1.71 $\times$, 2 $\times$, 1.33 $\times$, 2 $\times$ over the four baselines. On Open-Thoughts, it achieves the same 2 $\times$ improvement over chunked-prefill, NanoFlow and SGLang-PD, while Loongserve never meets SLO.

On ShareGPT, MUXWise, chunked-prefill, NanoFlow, and SGLang-PD all provide SLO guarantees at the beginning. SGLang-PD even achieves better TBT than MUXWise, as it statically reserves more compute. In contrast, MUXWise delivers shorter TTFT by reserving only best-fit SMs for decode. On OpenThoughts, LoongServe performs worse than the others, as it is designed for long-context workloads rather than requests with short inputs and long outputs.

NanoFlow outperforms chunked-prefill only on ShareGPT. On OpenThoughts, the system spends most of the time in the decode phase. Therefore, NanoFlow splits decode iterations to enable overlapping, leading to higher TBT than chunked-prefill. On LooGLE, it performs worse due to the small token budget used for long requests.

We also observe that SGLang-PD performs much worse on OpenThoughts and LooGLE than on ShareGPT. The causes differ across workloads. For OpenThoughts, since requests share little context, the system must still reserve slots for KV caches during prefill and decode. As the request rate

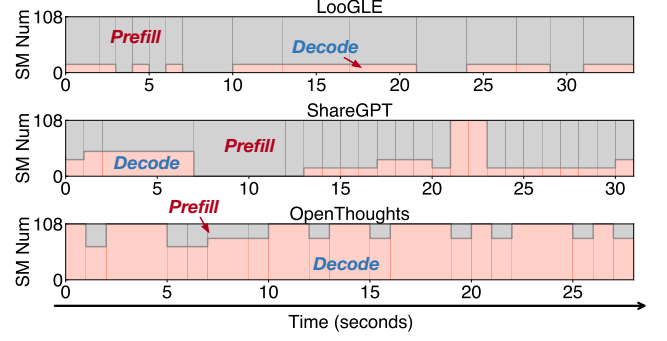


Figure 18. Change in compute partition between prefill and decode on LooGLE, ShareGPT, and OpenThoughts. Figures are sorted in descending order of prefill compute demand.

increases, prefill stalls once the KV cache pool runs out of space. For LooGLE, only four GPUs are available for prefill, causing requests to queue in the prefill instance.

4.3.1 Short Requests and Single GPU. Running Llama-8B on an A100 with ShareGPT, MUXWise improves goodput by 1.2 $\times$ over chunked-prefill while maintaining similar TBT. This is because even when chunking rarely happens, satisfying a strict TBT SLO still forces chunked-prefill to use a small token budget, limiting GPU utilization and peak goodput. Notably, real-world conversation inputs are becoming significantly longer (e.g., 1.2K and 2.3K tokens in two recent conversation traces from cloud vendors [35, 41], compared with 226 tokens in the older ShareGPT dataset). The conversation in our evaluation is a multi-turn real-world trace, whose average length approaches to 7.5K. This trend is driven by the widespread adoption of techniques such as RAG [24], which increase the effective model input length by appending retrieved sequences to the user input.

4.4 Ablation Study

4.4.1 Scheduling details of different tasks. We further evaluate MUXWise’s dynamic scheduling of compute partitions by extracting scheduling details from runtime serving in §4.3. As shown in Figure 18, MUXWise makes different scheduling decisions for different workloads. On LooGLE, most SMs are allocated to prefill, while on OpenThoughts, MUXWise allocates the majority of SMs to decode. Results on ShareGPT lie between LooGLE and OpenThoughts. Overall, however, more SMs are allocated to prefill on ShareGPT, since decode is typically memory-bound and does not require as many SMs as prefill. Notably, we use Figure 18 to show that different partitions are required for different workloads. They are relatively static because the request rate is stable. In real-world traces, the workload could be bursty. Experimental results show that, during a bursty interval in Figure 13, MUXWise activated all the six configurations within 30s.

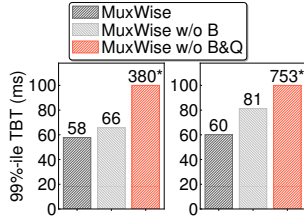


Figure 19. MuxWise with and without bubble-less multiplexing.

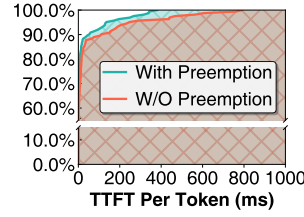


Figure 20. CDF of TTFT per token with and without preemption.

4.4.2 Effectiveness of Bubble-less Multiplex Engine.

As shown in Figure 9, bubbles commonly occur in the green context created for decode. In this experiment, we compare the TBT of MuxWise against its two variants. First, we disable layer-wise scheduling. Second, we further disable the query-based synchronization optimization. The workloads used are Tool&Agent under two different request rates.

Figure 19 presents the experimental results. As shown in the figure, disabling layer-wise execution slightly increases the TTFT of decode by approximately 10ms, which aligns with the typical kernel launch time for the prefill phase of Llama-70B. When query-based synchronization is further disabled, MuxWise suffers a significant degradation, 314ms for Llama-8B and 672ms for Llama-70B, due to frequent stalls waiting for the prefill phase to complete.

For further evaluation, we also collect the bubble ratio of MuxWise and chunked-prefill for the goodput results in Figure 15-a by profiling them with the NVIDIA Nsight Systems. The interval of the CUDA stream in the profiled timeline is treated as a bubble when it is not occupied by any GPU kernel. The bubble ratio is then defined as the proportion of all such bubbles in the compute stream. Since MuxWise has two active concurrent streams, we compute the bubble ratio for each stream and report their average as the final result. Notably, the bubble ratio is a temporal metric and does not reflect how GPU kernels utilize the parallel GPU resources they occupy.

MuxWise has a slightly higher bubble ratio (7.7% vs. 4.5%) due to its fine-grained kernel scheduling. These extra bubbles occur when the system is purely processing decode iterations and all prefill layers are completed. Fortunately, these bubbles do not degrade goodput, as there are no pending prefill launches and the decode iteration SLO is not violated. The reported GPU utilization in §4.2.3 also proves this.

4.4.3 Preemptive Scheduling for Long Request. We evaluate the benefit of the bubble-less multiplex engine for preemptive scheduling by mixing requests from ShareGPT and LooGLE (50% each). Requests are generated at a rate of 0.5 per second following a Poisson process. Figure 20 shows the CDF of TTFT per token with and without preemptive scheduling. As shown, MuxWise achieves a 1.96× speedup

on the 99%-ile TTFT per token, demonstrating that it can also be configured to support more advanced SLO-aware scheduling policies.

4.5 Overhead for Realizing PD-Multiplexing

Memory. MuxWise introduces some memory overhead by integrating GreenContext into existing serving systems. Creating a group of green contexts requires only 4MB, which is negligible compared to the total memory of modern GPUs. However, integrating it with CUDA Graph incurs a 6.2% overhead for both Llama-8B and Llama-70B on servers with 8 A100 or 8 H100 GPUs. This arises because the serving system records kernel launches for each decode-phase batch size into a CUDA Graph, consuming extra GPU memory. In MuxWise, there are six partition configurations in total, and each decode-phase compute partition created by GreenContext adds memory usage for all recorded batch sizes. Given the impressive performance gains of MuxWise, this overhead is acceptable.

Runtime. MuxWise splits the prefill phase into multiple prefill layers to enable bubble-less scheduling. This may introduce extra overhead due to fine-grained kernel launches. We conduct an experiment to compare full prefill launching with layer-wise launching, where the prefill phase is split into the finest granularity. Across various configurations with different batch sizes and context lengths, the total overhead remains within 1.5%.

5 Discussion

Generality of MuxWise. MuxWise generalizes to accelerators that support intra-process spatial sharing with lightweight dynamic adjustment, such as GreenContext [10] on NVIDIA GPUs (supported since the Pascal architecture) and `hipExtStreamCreateWithCUMask()` on AMD GPUs.

Contexts where MuxWise excels. MuxWise targets scenarios with strict SLO guarantees (e.g., a decode-phase SLO below 100ms). This is also the prevailing trend for achieving Model-as-a-Service in LLM serving. In this setting, MuxWise excels over existing works due to its efficient, fine-grained, and dynamic resource management between the prefill and decode phases. When the SLO target is loose or absent, such as in offline serving, MuxWise has no opportunity to outperform baselines such as chunked-prefill [55] or NanoFlow [54].

Large-scale deployment. While MuxWise is a single-instance optimization for high-goodput LLM serving, it can still benefit large-scale distributed deployments. In such deployments, MuxWise is complementary to disaggregated serving, as it optimizes each individual instance. Specifically, low-utilization decode instances could be replaced

with MuxWise instances to exploit idle resources via spatially multiplexing prefill. If prefill instance serves short requests—resulting in low utilization—or multiplexing decode on it does not violate TTFT SLO, it can be also utilized for higher efficiency. However, when prefill instances consistently handle long requests (e.g., using chunked pipeline parallelism [34]) or decode multiplexing violates TTFT SLO, MuxWise offers limited benefit.

6 Related Work

Multiplexing in LLM Serving. There are also prior works [16, 29] that multiplex the prefill and decode phases in LLM serving. WindServe [16] multiplexes prefill and decode using a normal CUDA stream, which leads to uncontrollable contention. It also does not address bubbles during scheduling. Our prototype implementation of WindServe shows that, on ShareGPT, MuxWise achieves a $1.61\times$ goodput improvement under a 50ms TBT SLO on an A100 with Llama-8B. Tropical [29] replaces the decode instance in disaggregated serving with temporally multiplexed prefill and decode. It launches a full prefill only when sufficient slack exists. When developing MuxWise, we implemented an enhanced temporal-only variant that splits prefill into layers to fit small slacks. It performs at least 20% worse than MuxWise because it cannot spatially leverage wasted resources. There are also two similar community works [18]. Semi-PD [18] utilizes MPS for multiplexing. MPS enables *inter-process* spatial sharing but requires process restarts to adjust SM allocations. Semi-PD mitigates this by introducing a resident process and two additional inference engines, which adds significant complexity to existing frameworks. Bullet [28] relies on `libsmctrl` [7] to control SM allocation. While Bullet claims that it can dynamically change the SMs allocated to each CUDA graph, our trials with Bullet’s open-sourced implementation show that the SM allocation for each CUDA graph does not change. This also aligns with the claim in `libsmctrl` [7] that it does not work with CUDA graph.

Compute management in LLM serving. There are two main approaches to compute management for improving system throughput under SLO constraints: disaggregation-based and fusion-based methods. On the one hand, DistServe [53] and Splitwise [32] disaggregate LLM serving into separate prefill and decode instances, while LoongServe [44] improves adaptability by enabling dynamic switching between the two at runtime. On the other hand, chunk-prefill [2] splits the prefill phase into chunks and fuses each chunk with a decode iteration for execution. However, disaggregation-based methods incur significant resource waste due to the coupled management of compute and memory, whereas fusion-based methods fail to fully maximize system throughput under SLO constraints. In contrast, our work decouples compute and memory management and maximizes goodput through spatial multiplexing.

Memory management in LLM serving. To improve system throughput in multi-turn or context-heavy LLM workloads, several systems propose memory management techniques. PagedAttention [23] introduces a paged memory pool to enable KV cache reuse between prefill and decode phases. Parrot [26] and SGLang [52] leverage context-aware caching to maximize reuse of KV segments across requests. MuxWise enhances these approaches by preserving memory sharing across phases and requests.

Execution time modeling. Performance modeling under spatial sharing is highly challenging, and prior efforts [22, 36, 48, 49] mainly focus on predicting interference for specific operators. GPUlet [9] uses linear regression with L1 cache utilization and DRAM bandwidth as input features to estimate performance interference among colocated operators. HSM [50] and GDP [20] also adopt linear regression based on low-level metrics in the simulator for operator slowdown prediction.

Compute partition techniques. Existing GPU partitioning techniques can be broadly categorized into time-sharing and space-sharing approaches. Time-sharing is typically implemented via API remoting [8, 27]. However, time-sharing alone is insufficient to meet MuxWise’s requirements, as the prefill and decode phases already interleave in a time-sharing manner. In contrast, NVIDIA provides MPS [13], MIG [12], and GreenContext [10] for spatial sharing. MPS and MIG support inter-process spatial multiplexing, while GreenContext [10] enables intra-process spatial multiplexing with precise SM partition [11]. MuxWise builds on GreenContext to implement its PD multiplexing approach.

7 Conclusion

LLM services requires high goodput, yet existing serving systems struggle due to various deficiencies. To address these issues, we present MuxWise, an LLM serving framework with high goodput. MuxWise leverages a promising new serving paradigm, intra-GPU PD multiplexing, to achieve more flexible compute management for prefill and decode phases in LLM serving. Experiments show that MuxWise improves goodput by $2.2\times$ on average over state-of-the-art baselines. Despite the notable performance improvement, MuxWise also introduces a simple yet effective design for current LLM serving systems. We plan to open-source MuxWise after publication.

Acknowledgments

This work is partially sponsored by the National Key Research and Development Program of China (2024YFB4505700), National Natural Science Foundation of China (62232011) and Natural Science Foundation of Shanghai Municipality (24ZR1430500). We thank the anonymous reviewers for their constructive feedback and suggestions.

A Artifact Appendix

A.1 Abstract

MuxWise is an LLM serving framework adopting intra-GPU prefill-decode multiplexing, which is built on the top of SGLang[52]. We provide the source code of MuxWise and scripts to reproduce comparison of chunked-prefill. This appendix includes instructions for reproducing similar data in Figure 16 and Figure 17.

A.2 Artifact check-list (meta-information)

- **Model:** CodeLlama-34b-Instruct-hf.
- **Data set:** ShareGPT [4] and LooGLE [25].
- **Hardware:**
NVIDIA H200 NVL (140 GB, 132 SMs)
NVIDIA driver: 580.65.06 (must be greater than 570)
- **Experiments:** This appendix provides instructions for comparing 99%-ile TTFT and 99%-ile TBT MuxWise between MuxWise and chunked-prefill under various workload.
- **Metrics:** 99%-ile TTFT, 99%-ile TBT
- **Output:** Jsonl files containing metrics from MuxWise and chunked-prefill with different chunk size.
- **How much disk space required (approximately)?:** Approximately 200GB
- **How much time is needed to prepare workflow (approximately)?:** About 10 minutes to build from source code.
- **How much time is needed to complete experiments (approximately)?:** About 2 hours for ShareGPT workload and 4 hours for LooGLE workload.
- **Publicly available?:** Yes.

A.3 Description

A.3.1 How to access. The source code of MuxWise is available for download on Zenodo: <https://zenodo.org/records/18062118>. The pre-built Docker image can be found in: https://hub.docker.com/layers/combathhhhhh/pdmux/sglpr_torch2.6_bench

A.3.2 Hardware dependencies. Requires an x86-64 Linux host with at least 200 GB of free disk space, and an NVIDIA H200 NVL GPU (140 GB, 132 SMs).

A.3.3 Software dependencies. NVIDIA driver: 580.65.06 (must be greater than 570).

A.3.4 Data sets. ShareGPT: chatbot tasks, with an average input length of 226 and average output length of 195.

LooGLE: long-context understanding tasks, with an average input length of 30k and average output length of 15.

A.3.5 Models. CodeLlama-34b-Instruct-hf.

A.4 Installation

Please follow the instructions below, which are adapted from our GitHub repository (https://github.com/ykcombat/sglang/tree/slo_config):

1. Clone the repository and switch to the slo_config branch

```
git clone https://github.com/ykcombat/sglang.git
cd slang
git checkout slo_config
```

```
# 2. Build SGLang
pip install --upgrade pip
pip install -e "python"
```

A.5 Experiment workflow

Our experiments focus on comparison between MuxWise and chunked-prefill.

1. Download the required LLM model(CodeLlama-34b-Instruct-hf) to /workspace/data.
2. Start MuxWise or chunked-prefill server. You can change the environment variable \$CHUNK_SIZE to start chunked-prefill server with different token budgets.

```
# 1. Start MuxWise Server
./start_pdmux.sh
```

```
# 2. Start Chunked-prefill Server
./start_chunk.sh
```

3. Start evaluating in another terminal.

```
# 1. Evaluate MuxWise on ShareGPT and LooGLE
./bench_pdmux.sh
```

```
# 2. Evaluate Chunked-prefill on ShareGPT and LooGLE
./bench_chunk.sh
```

A.6 Evaluation and expected results

When all experiments done, you will obtain jsonl files containing detailed metrics under /workspace/sglang. To visualize the results, run plot.ipynb; this will generate figures similar to the reference plot provided at /workspace/sglang/H200_result.png. Please note that results may vary depending on the specific hardware used. You can refer to https://github.com/ykcombat/sglang/blob/slo_config/README.md for more information.

A.7 Notes

When serving different workloads, different configurations are used, which can be found in our repository.

References

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming throughput-latency tradeoff in LLM inference with sarathi-serve. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) (OSDI'24). USENIX Association, USA, Article 7, 18 pages.

- [2] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. 2023. SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills. arXiv:2308.16369 (Aug. 2023). arXiv:2308.16369 [cs]
- [3] Michael Andersch, Greg Palmer, Ronny Krashinsky, Nick Stam, Vishal Mehta, Gonzalo Brito, and Sridhar Ramaswamy. 2025. NVIDIA Hopper Architecture In-Depth – Thread block clusters. https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/#thread_block_clusters. Accessed 2026-01-10.
- [4] anon8231489123. 2023. ShareGPT Vicuna Unfiltered – Cleaned Split (v3). https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered/resolve/main/ShareGPT_V3_unfiltered_cleaned_split.json. Accessed: 2025-04-16.
- [5] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, and et al. 2023. Qwen Technical Report. arXiv:2309.16609 (Sept. 2023). arXiv:2309.16609 [cs] doi:10.48550/arXiv.2309.16609
- [6] Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, et al. 2024. LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. *arXiv preprint arXiv:2412.15204* (2024).
- [7] Joshua Bakita and James H Anderson. 2023. Hardware compute partitioning on NVIDIA GPUs. In *2023 IEEE 29th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 54–66.
- [8] Quan Chen, Hailong Yang, Jason Mars, and Lingjia Tang. 2016. Baymax: QoS Awareness and Increased Utilization for Non-Preemptive Accelerators in Warehouse Scale Computers. In *Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, Atlanta Georgia USA, 681–696. doi:10.1145/2872362.2872368
- [9] Seungbeom Choi, Sunho Lee, Yeonjae Kim, Jongse Park, Youngjin Kwon, and Jaehyuk Huh. 2022. Serving heterogeneous machine learning models on {Multi-GPU} servers with {Spatio-Temporal} sharing. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. 199–216.
- [10] NVIDIA Corporation. 2025. CUDA Driver API: Green Contexts. https://docs.nvidia.com/cuda/cuda-driver-api/group__CUDA_GREEN_CONTEXTS.html. Accessed: 2025-03-29.
- [11] NVIDIA Corporation. 2025. CUDA Runtime API: Stream Management. https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART_STREAM.html. Accessed: 2025-03-30.
- [12] NVIDIA Corporation. 2025. Multi-Instance GPU (MIG). <https://www.nvidia.com/en-sg/technologies/multi-instance-gpu/>. Accessed: 2025-03-30.
- [13] NVIDIA Corporation. 2025. *Multi-Process Service*. Version 570.
- [14] NVIDIA Corporation. 2025. NVIDIA Dynamo: A Datacenter Scale Distributed Inference Serving Framework. <https://github.com/ai-dynamo/dynamo>. Accessed: 2025-04-07.
- [15] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, and et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 (Aug. 2024). arXiv:2407.21783
- [16] Jingqi Feng, Yukai Huang, Rui Zhang, Sicheng Liang, Ming Yan, and Jie Wu. 2025. WindServe: Efficient Phase-Disaggregated LLM Serving with Stream-based Dynamic Scheduling. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. Association for Computing Machinery, New York, NY, USA, 1283–1295. doi:10.1145/3695053.3730999
- [17] Etash Guha, Ryan Marten, Sedrick Keh, Negin Raoof, Georgios Smyrnis, Hritik Bansal, Marianna Nezhurina, Jean Mercat, Trung Vu, Zayne Sprague, Ashima Suvarna, Benjamin Feuer, Liangyu Chen, Zaid Khan, Eric Frankel, Sachin Grover, Caroline Choi, Niklas Muennighoff, Shiye Su, Wanjia Zhao, John Yang, Shreyas Pimpalgaonkar, Kartik Sharma, Charlie Cheng-Jie Ji, Yichuan Deng, Sarah Pratt, Vivek Ramanujan, Jon Saad-Falcon, Jeffrey Li, Achal Dave, Alon Albalak, Kushal Arora, Blake Wulfe, Chinmay Hegde, Greg Durrett, Sewoong Oh, Mohit Bansal, Saa-dia Gabriel, Aditya Grover, Kai-Wei Chang, Vaishaal Shankar, Aaron Gokaslan, Mike A. Merrill, Tatsunori Hashimoto, Yejin Choi, Jenia Jitsev, Reinhard Heckel, Maheswaran Sathiamoorthy, Alexandros G. Dimakis, and Ludwig Schmidt. 2025. OpenThoughts: data recipes for reasoning models. doi:10.48550/arXiv.2506.04178 arXiv:2506.04178 [cs].
- [18] Ke Hong, Lufang Chen, Zhong Wang, Xiuhong Li, Qiuli Mao, Jianping Ma, Chao Xiong, Guanyu Wu, Buhe Han, Guohao Dai, Yun Liang, and Yu Wang. 2025. semi-PD: Towards Efficient LLM Serving via Phase-Wise Disaggregated Computation and Unified Storage. arXiv:2504.19867 [cs.CL] <https://arxiv.org/abs/2504.19867>
- [19] Anysphere Inc. 2025. Cursor: The AI Code Editor. <https://www.cursor.com/>. Accessed: 2025-04-05.
- [20] Magnus Jahre and Lieven Eeckhout. [n. d.]. Gdp: Using dataflow properties to accurately estimate interference-free performance at runtime. In *IEEE International Symposium on High Performance Computer Architecture (HPCA 2018)*. 296–309.
- [21] Aditya K. Kamath, Ramya Prabhu, Jayashree Mohan, Simon Peter, Ramachandran Ramjee, and Ashish Panwar. 2025. POD-attention: unlocking full prefill-decode overlap for faster LLM inference. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 897–912. doi:10.1145/3676641.3715996
- [22] Sejin Kim and Yoonhee Kim. 2022. K-Scheduler: Dynamic Intra-SM Multitasking Management with Execution Profiles on GPUs. *Cluster Computing* 25, 1 (Feb. 2022), 597–617. doi:10.1007/s10586-021-03429-7
- [23] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. ACM, Koblenz Germany, 611–626. doi:10.1145/3600006.3613165
- [24] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [25] Jiaqi Li, Mengmeng Wang, Zilong Zheng, and Muhan Zhang. 2024. LooGLE: Can Long-Context Language Models Understand Long Contexts? arXiv:2311.04939 [cs.CL] <https://arxiv.org/abs/2311.04939>
- [26] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient Serving of LLM-based Applications with Semantic Variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 929–945.
- [27] Yu-Shiang Lin, Chun-Yuan Lin, Che-Rung Lee, and Yeh-Ching Chung. 2019. qcuda: Gpgpu virtualization for high bandwidth efficiency. In *2019 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. IEEE, 95–102.
- [28] Zejia Lin, Hongxin Xu, Guanyi Chen, Zhiguang Chen, Yutong Lu, and Xianwei Zhang. 2025. Boosting LLM Serving through Spatial-Temporal GPU Resource Sharing. arXiv:2504.19516 [cs.DC] <https://arxiv.org/abs/2504.19516>
- [29] Jinming Ma, Jiefei Chen, Xiuhong Li, Jiangfei Duan, Haojie Duanmu, Xingcheng Zhang, Chao Yang, and Dahua Lin. 2025. *Tropical: Enhancing SLO Attainment in Disaggregated LLM Serving via SLO-Aware Multiplexing*. IEEE Press. <https://doi.org/10.1109/DAC63849.2025.11132617>
- [30] OpenAI. 2025. ChatGPT. <https://chatgpt.com/>. Accessed: 2025-04-05.
- [31] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein,

- Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, and et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Curran Associates Inc., Red Hook, NY, USA, 8026–8037.
- [32] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 118–132. doi:10.1109/ISCA59077.2024.00019
- [33] Zhenting Qi, Mingyuan Ma, Jiahang Xu, Li Lina Zhang, Fan Yang, and Mao Yang. 2024. Mutual Reasoning Makes Smaller LLMs Stronger Problem-Solvers. arXiv:2408.06195 (Aug. 2024). arXiv:2408.06195
- [34] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — a KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. 155–170.
- [35] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2025. DynamoLLM: designing LLM inference clusters for performance and energy efficiency. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 1348–1362. doi:10.1109/HPCA61900.2025.00102 ISSN: 2378-203X.
- [36] Foteini Strati, Xianzhe Ma, and Ana Klimovic. 2024. Orion: Interference-Aware, Fine-Grained GPU Sharing for ML Applications. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys '24)*. Association for Computing Machinery, New York, NY, USA, 1075–1092. doi:10.1145/3627703.3629578
- [37] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971 (Feb. 2023). arXiv:2302.13971 doi:10.48550/arXiv.2302.13971
- [38] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288 [cs.CL] <https://arxiv.org/abs/2307.09288>
- [39] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention Is All You Need. In *Advances in Neural Information Processing Systems*. NeurIPS, Long Beach, CA, USA. arXiv:1706.03762 doi:10.48550/arXiv.1706.03762
- [40] vLLM Team. [n. d.]. CUDA Graphs — vLLM Design Documentation. https://docs.vllm.ai/en/stable/design/cuda_graphs/. Accessed: 2026-01-09.
- [41] Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyuan Yu, and Haibo Chen. 2025. KVCache cache in the wild: characterizing and optimizing KVCache cache at a large cloud provider. 465–482. <https://www.usenix.org/conference/atc25/presentation/wang-jiahao>
- [42] Zhibin Wang, Shipeng Li, Yuhang Zhou, Xue Li, Zhonghui Zhang, Nguyen Cam-Tu, Rong Gu, Chen Tian, Guihai Chen, and Sheng Zhong. 2025. Revisiting Service Level Objectives and System Level Metrics in Large Language Model Serving. arXiv:2410.14257 [cs.LG] <https://arxiv.org/abs/2410.14257>
- [43] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H Chi, Quoc V Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [44] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. LoongServe: Efficiently Serving Long-Context Large Language Models with Elastic Sequence Parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP '24)*. Association for Computing Machinery, New York, NY, USA, 640–654. doi:10.1145/3694715.3695948
- [45] Jiayi Yao, Hanchen Li, Yuhang Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *Proceedings of the Twentieth European Conference on Computer Systems (Rotterdam, Netherlands) (EuroSys '25)*. Association for Computing Machinery, New York, NY, USA, 94–109. doi:10.1145/3689031.3696098
- [46] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. In *Eighth Conference on Machine Learning and Systems*.
- [47] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 521–538.
- [48] Shulai Zhang, Quan Chen, Weihao Cui, Han Zhao, Chunyu Xue, Zhen Zheng, Wei Lin, and Minyi Guo. 2025. Improving GPU Sharing Performance through Adaptive Bubbleless Spatial-Temporal Sharing. In *Proceedings of the Twentieth European Conference on Computer Systems (ACM Conferences)*. 573–588. doi:10.1145/3689031.3696070
- [49] Wei Zhang, Weihao Cui, Kaihua Fu, Quan Chen, Daniel Edward Mawhirther, Bo Wu, Chao Li, and Minyi Guo. 2019. Laius: Towards Latency Awareness and Improved Utilization of Spatial Multitasking Accelerators in Datacenters. In *Proceedings of the ACM International Conference on Supercomputing (ICS '19)*. Association for Computing Machinery, New York, NY, USA, 58–68. doi:10.1145/3330345.3330351
- [50] Xia Zhao, Magnus Jahre, and Lieven Eeckhout. [n. d.]. HSM: A Hybrid Slowdown Model for Multitasking GPUs. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2022)*. 1371–1385.
- [51] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P. Xing, Joseph E. Gonzalez, and Ion Stoica. 2022. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 559–578.
- [52] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: efficient execution of structured language model programs. In *Proceedings of the 38th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS '24)*. Curran Associates Inc., Red Hook, NY, USA, Article 2000, 27 pages.
- [53] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model

- Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 193–210.
- [54] Kan Zhu, Yufei Gao, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Zihao Ye, Keisuke Kamahori, Chien-Yu Lin, Ziren Wang, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. 2025. NanoFlow: Towards Optimal Large Language Model Serving Throughput. 749–765. <https://www.usenix.org/conference/osdi25/presentation/zhu-kan>
- [55] Simiao Zuo, Xiaodong Liu, Jian Jiao, Young Jin Kim, Hany Hassan, Ruofei Zhang, Tuo Zhao, and Jianfeng Gao. 2022. Taming Sparsely Activated Transformer with Stochastic Experts. arXiv:2110.04260 (Feb. 2022). arXiv:2110.04260 [cs]